

RECAP C++ DEEL I

functies, pointers, references



PROGRAMMA

1. Recap
2. Quiz
3. Aan de slag
4. Oefenproject `software/cpp/recap_1`
5. Verder kijken

RECAP

Al gehad

- S2: functies, by value / by reference, `vector`, structs, klassen
- S3: heap, `new` / `delete`, levensduur, rule of three, smart pointers
- Vorige week: devcontainer, CMake, debugger

Vandaag: bij elke variabele, pointer of reference dezelfde drie vragen

- Waar staat mijn data? Stack of heap?
- Wie is er eigenaar van?
- Hoe lang leeft het?

Komt later

- STL & lambda's: iterators zijn pointers, captures zijn by value / by reference
- Smart pointers & design patterns: geheugenbeheer zonder `new` / `delete`
- Pointers, references & UB: de diepte in

QUIZ

WAT PRINT DIT? (1)

```
1 void inc(int x) { x++; }  
2  
3 int main() {  
4     int a = 5;  
5     inc(a);  
6     std::cout << a;  
7 }
```

5

`x` is een *kopie* van `a`.

WAT PRINT DIT? (2)

```
1 void inc(int& x) { x++; }  
2  
3 int main() {  
4     int a = 5;  
5     inc(a);  
6     std::cout << a;  
7 }
```

6

`x` is een `alias` van `a` : een andere naam voor hetzelfde geheugen.

WAT PRINT DIT? (3)

```
1  int main() {  
2      int a = 5;  
3      int* p = &a;  
4      *p = 7;  
5      int& r = a;  
6      r++;  
7      std::cout << a;  
8  }
```

8

`p` en `r` verwijzen allebei naar `a`. Verschil: `p` is een eigen pointer van `a`, dus eigen adres, `r` is alleen een andere naam.

Gebruik alleen pointers als het nodig is!

WAT PRINT DIT? (4)

```
1  int main() {  
2      int arr[] = {10, 20, 30};  
3      int* p = arr;  
4      p++;  
5      std::cout << *p + *(p + 1);  
6  }
```

50

`p++` springt één **element** verder, niet één byte. `p[i]` is `*(p + i)` .

WAT PRINT DIT? (5)

```
1  int main() {  
2      int* p = new int(5);  
3      int* q = p;  
4      delete p;  
5      std::cout << *q;  
6  }
```

Undefined behaviour. Use after free. Print misschien 5 , misschien iets anders, misschien een crash, misschien velociraptors.

WAT PRINT DIT? (6)

```
1  int& getValue() {  
2      int x = 10;  
3      return x;  
4  }  
5  
6  int main() {  
7      int y = getValue();  
8      std::cout << y;  
9  }
```

Undefined behaviour. `x` leeft op de stack van `getValue` en bestaat niet meer. De compiler *waarschuwt* hier; lees je warnings.

WAT PRINT DIT? (7)

```
1  int main() {  
2      const int x = 5;  
3      int* p = &x;  
4      *p = 6;  
5      std::cout << x;  
6  }
```

Compileert niet. `&x` is een `const int*`, dat past niet in een `int*`. `const` is een belofte die de compiler voor je bewaakt.

WAT PRINT DIT? (8)

```
1 void f(int a[]) {  
2     std::cout << sizeof(a);  
3 }  
4  
5 int main() {  
6     int arr[10];  
7     std::cout << sizeof(arr) << " ";  
8     f(arr);  
9 }
```

40 8

een array **vervalt (decayed)** tot een **pointer** zodra je hem doorgeeft. In `f` is `a` gewoon een `int*`.

AAN DE SLAG

OEFENPROJECT `software/cpp/recap_1`

Bestand	Wat
1 <code>src/01_funcities.cc</code>	<code>clamp</code> , <code>swap</code> by reference en by pointer
2 <code>src/02_pointers.cc</code>	<code>findMax</code> , <code>findValue</code> , <code>reverse</code> met <code>[begin, end)</code>
3 <code>src/03_bughunt.cc</code>	acht verstopte bugs : vind ze met warnings, sanitizers en debugger

Opstarten: `git pull` , open de map `software/cpp/recap_1` in VSCode, *Reopen in Container*, CMake Tools: target kiezen, Build, Run of `F5` .

VERDER KIJKEN

- [Back to Basics: Pointers and Memory — Ben Saks \(CppCon\)](#)
- [cppreference: Undefined behavior](#)
- [AddressSanitizer](#)

\0